

****Project Title**:** Tic Tac Toe Game
By Cory Simonsen

****1. Introduction****

The Tic Tac Toe Game is a Python-based implementation of the classic two-player strategy game. This project showcases several technical features, including player versus computer gameplay with AI, advanced algorithms like minimax with alpha-beta pruning, dynamic board sizing, and a text-based interface.

****2. Design and Implementation****

****Design**:** The project follows an Object-Oriented Programming (OOP) approach and comprises three primary modules: ``game_model.py``, ``game_view.py``, and ``main.py``.

- ``game_model.py``: This module encapsulates the game's core data structures and logic. It defines classes for the game board, player, and game data. The board is represented as a 2D list, and it includes the logic for win or draw conditions.

- ``game_view.py``: Responsible for the user interface and interaction, this module hosts the ``TicTacToe`` class. It manages rendering the game board, obtaining user input, and controlling the game flow.

- ``main.py``: The entry point of the game initializes the necessary components and orchestrates the main game loop.

****Implementation**:** The game boasts several noteworthy technical implementations:

- Player versus computer gameplay: Players can select their symbol (X or O) and decide whether they want to make the first move. The computer opponent employs a sophisticated minimax algorithm with alpha-beta pruning for intelligent decision-making.

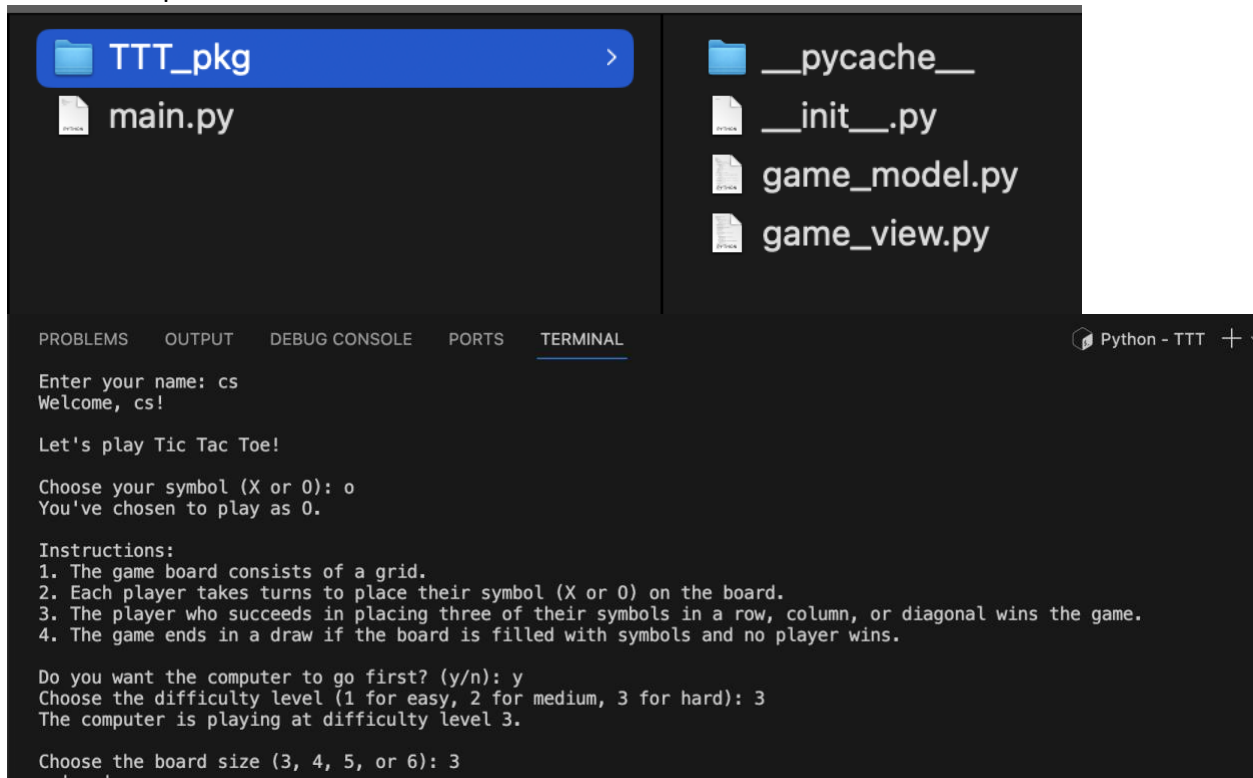
- Variable board sizes: Players can choose from a range of board sizes, from the classic 3x3 grid to larger 6x6 grids, adding complexity and challenge.

- User-friendly interface: The game features clear instructions, guides players through symbol selection, and presents the game board visually.

****Modifications and Challenges**:** Key challenges during implementation included optimizing the computer AI for larger board sizes. To address this, we fine-tuned the minimax algorithm and introduced alpha-beta pruning for enhanced performance. Codebase optimization focused on modularity and readability.

****Screenshots**:**

- Initial Setup:



```
Enter your name: cs
Welcome, cs!

Let's play Tic Tac Toe!

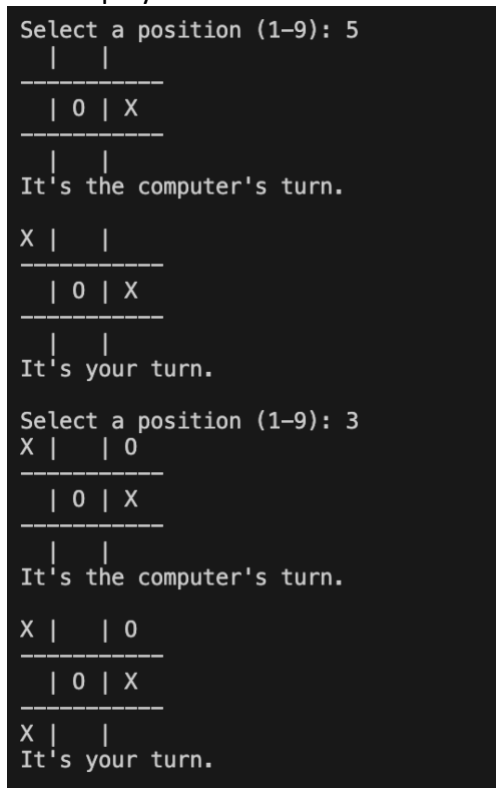
Choose your symbol (X or O): o
You've chosen to play as O.

Instructions:
1. The game board consists of a grid.
2. Each player takes turns to place their symbol (X or O) on the board.
3. The player who succeeds in placing three of their symbols in a row, column, or diagonal wins the game.
4. The game ends in a draw if the board is filled with symbols and no player wins.

Do you want the computer to go first? (y/n): y
Choose the difficulty level (1 for easy, 2 for medium, 3 for hard): 3
The computer is playing at difficulty level 3.

Choose the board size (3, 4, 5, or 6): 3
```

- Gameplay:



```
Select a position (1-9): 5
|  | 
-----
| O | X
-----
|  | 
It's the computer's turn.
X |  | 
-----
| O | X
-----
|  | 
It's your turn.
Select a position (1-9): 3
X |  | O
-----
| O | X
-----
|  | 
It's the computer's turn.
X |  | O
-----
| O | X
-----
X |  | 
It's your turn.
```

```

  | | | | |
-----
X | | 0 | | | 0
It's your turn.

Select a position (1-36): 19
X | 0 | 0 | | | X
-----
X | | | 0 | | 
-----
X | | | | | 
-----
X | 0 | | | | 
-----
  | | | | | 
-----
X | | 0 | | | 0
It's the computer's turn.

X | 0 | 0 | | | X
-----
X | | | 0 | | 
-----
X | | | | | 
-----
X | 0 | | | | 
-----
0 | | | | | 
-----
X | | 0 | | | 0
It's your turn.

Select a position (1-36): █

```

- Game Over:

```

Select a position (1-36): 24
X | 0 | 0 | 0 | X | X
-----
X | 0 | 0 | 0 | X | X
-----
X | X | 0 | X | 0 | 0
-----
X | 0 | X | 0 | X | X
-----
0 | 0 | X | 0 | 0 | X
-----
X | 0 | 0 | X | X | 0
It's a draw!
Do you want to play again? (y/n): n
Bye! Thanks for Playing!
Corys-Air:TTT csair$ █

```

****3. Conclusions****

- ****Technical Learning****: This project provided practical experience in AI implementation, advanced algorithm design, and user interface development. It allowed for skill development in problem-solving and coding efficiency.

- **Technical Highlights**: Notable features include adjustable board sizes, intelligent computer AI, and modular code architecture, promoting maintainability and extensibility.
- **Technical Shortcomings**: While the AI performs well, further optimization for larger boards remains a consideration. A graphical user interface could enhance user engagement.
- **Technical Hindsight**: In retrospect, exploring graphical libraries like Pygame could have been advantageous for a visually appealing interface. Future endeavors might involve exploring other open source libraries.
- **Technical Future**:
 - **Graphical User Interface (GUI)**: Implementing a GUI using libraries like Pygame or tkinter would make the game more visually appealing and user-friendly, attracting a broader audience.
 - **Multiplayer Mode**: Incorporate a multiplayer mode that allows two human players to compete against each other, either locally or over a network.
 - **Online Play**: Extend the game to enable online multiplayer matches, offering a competitive experience with players from around the world.
 - **AI Optimization**: Continue refining the AI's decision-making process, especially for larger board sizes, to maintain an optimal player experience.
 - **Customization**: Allow players to customize game elements, such as board size, symbols, and AI difficulty levels, to cater to different preferences.
 - **Statistics and Leaderboards**: Implement features to track player statistics and maintain leaderboards to encourage competition and engagement.

By investing some time to experiment, the Tic Tac Toe Game can evolve into a more comprehensive and engaging gaming experience, appealing to a wider audience of players and provide a platform for future enhancements.